M2S-04: Step 4 — Prepare: Readiness and Pre-Migration

> **Series:** M2S — Managed to SaaS Migration | **Notebook:** 4 of 9 | **Phase:** Upgrade | **Step:** Prepare | **Created:** March 2026 | **Last Updated:** 10/01/2026

With the target architecture designed, it is time to prepare everything needed for migration execution. This step ensures your SaaS tenant is provisioned, identity is configured, ActiveGates are deployed in parallel, and rollback procedures are tested—so that when you flip the switch in Step 5, there are no surprises.

> **M2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan:** 1. Discover | 2. Strategize | 3. Design
>
> **Upgrade: 4. Prepare** | 5. Execute | 6. Integrate
>
> **Run:** 7. Enable | 8. Expand | 9. Optimize

Table of Contents

1. Licensing and Contract Alignment
2. SaaS Tenant Provisioning
3. SSO and Identity Setup
4. ActiveGate Provisioning
5. SaaS Upgrade Assistant Setup
6. Network Zones in the SaaS Tenant
7. Configuration Freeze
8. Rollback Procedure
9. Readiness Checklist

Prerequisites

Before starting this notebook, you should have:

Requirement	Description
Steps 1–3 completed	Discovery inventory, migration strategy, and target architecture designed
Dynatrace account team contact	For licensing coordination and tenant provisioning
Identity provider access	Admin access to your SAML IdP (Azure Entra, Okta, etc.)
Infrastructure access	Ability to deploy new ActiveGate VMs/instances
Network changes approved	Firewall rules from Design step implemented or in progress
Managed cluster version	Align to the same *major* version as the target SaaS tenant (docs state no fixed minimum — verify the current requirement)

Learning Objectives

By the end of this notebook, you will:

- Understand the licensing considerations for a dual-run migration period
- Provision and configure a SaaS tenant in the correct region
- Set up SAML SSO and initial IAM groups and policies
- Deploy SaaS-connected ActiveGates in parallel with Managed AGs
- Install the SaaS Upgrade Assistant and import a Managed configuration archive
- Confirm or create network zones in the SaaS tenant
- Implement a configuration freeze on the Managed environment
- Document and test a rollback procedure
- Complete the readiness checklist before proceeding to Execute

---

Order of Operations — You Are Here

This notebook covers operations **1-3** of the 11-step Order of Operations (see **M2S-02: Step 2 — Strategize**) defined in Step 2:

#	Operation	Status
1	**Assess — Inventory current environment**	**This notebook**
2	**Provision — SaaS tenant and access (SSO)**	**This notebook**
3	**Install — New ActiveGates in parallel**	**This notebook**
4	Migrate — Configuration and integrations	Step 5: Execute
5	Rebuild — Dashboards, zones, alerts	Step 5: Execute
6	Redirect — OneAgents to SaaS	Step 5: Execute
7	Reconnect — Integrations and extensions	Step 6: Integrate
8	Migrate — Remaining config and integrations	Step 6: Integrate
9	Validate — Data flow and performance	Step 9: Optimize
10	Cutover — Full switch to SaaS	Step 9: Optimize
11	Decommission — Managed environment	Step 9: Optimize

1. Licensing and Contract Alignment

Before any technical work begins, coordinate with your Dynatrace account team to ensure licensing covers the migration period.

1.1 Dual-Run Licensing

During migration, both the Managed environment and the SaaS tenant are active simultaneously. This **dual-run period** typically lasts 2–4 weeks, depending on the size and complexity of your environment.

Consideration	Detail
Overlap period	Plan for 2–4 weeks of parallel operation
DPS model	SaaS uses Dynatrace Platform Subscription (DPS)
Managed licensing	Existing Managed license remains active during migration
Budget impact	Account for additional cost during overlap
Account team	Coordinate timing so both licenses are active simultaneously

> **Important:** Do not let the Managed license expire before all agents are migrated. If the Managed license lapses during migration, OneAgents still connected to Managed will stop reporting.
>
> **Complimentary overlap licensing:** In community practice, Dynatrace commonly provides complimentary parallel SaaS licensing to cover the migration overlap (often cited as up to ~6 months) — this is not a contractual entitlement, so confirm the scope and duration with your account team when you plan the dual-run window above.

1.2 DPS (Dynatrace Platform Subscription)

SaaS uses the DPS consumption model. Key differences from Managed licensing:

Aspect	Managed	SaaS (DPS)
Billing unit	Host units (HU)	DPS credits
Consumption model	Fixed capacity	Consumption-based
Data retention	Managed by customer	Configurable per data type
Grail storage	N/A (Managed uses Cassandra)	Included in DPS

1.3 Timeline Coordination

Milestone	Action
T-4 weeks	Confirm dual-run licensing with account team

Milestone	Action
T-2 weeks	SaaS tenant provisioned and accessible
T-1 week	All preparation complete (this notebook)
T-0	Begin agent migration (Step 5)
T+2–4 weeks	Complete migration, decommission Managed

---

2. SaaS Tenant Provisioning

Your Dynatrace account team provisions the SaaS tenant. You must provide key decisions before provisioning begins.

2.1 Provisioning Decisions

Decision	Options	Impact
Cloud provider	AWS, Azure or Google Cloud	Determines underlying infrastructure and the regions on offer
Data residency region	One of the regions listed on the data-security-controls page	**Treat as permanent** — moving means a new tenant
Azure Native	Azure Native Dynatrace Service (if applicable)	Simplified billing via Azure Marketplace

> **Critical:** The data residency region is permanent. Confirm your choice with compliance, legal, and security teams before requesting provisioning. Moving to a different region later requires a new tenant and full re-migration.

2.2 Azure Native Dynatrace Service

If your organization uses Azure, consider the Azure Native Dynatrace Service:

Benefit	Detail
Marketplace billing	Dynatrace costs appear on your Azure invoice
Simplified provisioning	Create Dynatrace resources directly in Azure portal
Native integration	Automatic log and metric forwarding from Azure resources
Single sign-on	Leverage existing Azure Entra ID without separate SAML config

2.3 Initial Tenant Configuration

After the tenant is provisioned, configure these baseline settings:

Setting	Location	Value
Timezone	Settings > Preferences	Match your primary operations timezone
Currency	Settings > Preferences	Match your reporting currency
Data retention	Settings > Data privacy > Data retention	Set per data type (logs, metrics, spans)
Session timeout	Account Management > Security	Align with your organization's policy

> **Decide Grail bucket strategy before any logs flow — it is largely irreversible.** The **Data retention** row above sets *default* per-type retention. For a log-heavy migration the higher-leverage decision is your **custom Grail bucket layout**: which log sources land in which buckets, each bucket's retention, and its query-billing model. Records cannot be moved between buckets after ingest, and a bucket's name and class (live/historic) are fixed at creation, while retention and the query-billing model can be changed later — so this must be designed *before* Step 5 redirects high-volume log sources (otherwise everything lands in `default_logs` and cannot be re-bucketed later). Design it now against **ORGNZ-03: Bucket Strategy and Design** and **OPLOGS-04: Buckets & Data Governance**, and plan to route logs via the OpenPipeline **Storage** stage.

2.4 Verify Tenant Access

Confirm all migration team members can access the new tenant:

Endpoint	Test
`https://{tenant-id}.live.dynatrace.com`	Browser login succeeds
`https://{tenant-id}.apps.dynatrace.com`	Apps platform loads
`https://{tenant-id}.live.dynatrace.com/api/v1/time`	API responds with server time

```
# Verify API access
curl -s "https://{tenant-id}.live.dynatrace.com/api/v1/time" \
  -H "Authorization: Api-Token {token}" | python3 -m json.tool
```

2.5 Request Feature Flags (if needed)

Some SaaS capabilities are gated behind **feature flags** that Dynatrace enables per tenant on request. If your target design depends on a feature that is not visible in the freshly provisioned tenant, raise a Dynatrace Support request during preparation so it is enabled before Step 5 (Execute) — do not wait until cutover to discover a gap.

---

3. SSO and Identity Setup

Dynatrace documents SAML 2.0 federation for SaaS single sign-on. If your Managed environment uses LDAP, this is the point where you transition to SAML.

3.1 SAML 2.0 Configuration Requirements

Requirement	Detail
Protocol	SAML 2.0
IdP signing	Must sign the **entire SAML message** (not just the assertion)
Name ID	Email address format recommended

Requirement	Detail
Attributes	First name, last name, email, groups
SP Entity ID	Provided by Dynatrace during SAML setup
ACS URL	Provided by Dynatrace during SAML setup

> **Critical:** If the IdP signs only the SAML assertion (not the full message), sign-in fails with `400 Bad Request`. **Microsoft Entra ID signs only the assertion by default** for most gallery applications — set the enterprise application's signing option to **Sign SAML response and assertion**. For other IdPs (Okta, PingFederate, ADFS), verify the equivalent setting explicitly.
>
> **Sources:** SAML (DT docs) — *"The entire SAML message must be signed (signing only SAML assertions is insufficient and generates a 400 Bad Request response)."*; Advanced certificate signing options in a SAML token (Microsoft Learn) — *"default option set for most of the gallery applications"*.

3.2 Azure Entra ID Configuration

For organizations using Azure Entra ID as their IdP:

Step	Action
1	Create Enterprise Application for Dynatrace SSO
2	Configure SAML settings with SP Entity ID and ACS URL; set **Signing Option** to *Sign SAML response and assertion*
3	Map attributes: UPN, email, first name, last name
4	**Filter group claims** to Dynatrace-related groups only
5	Assign users and groups to the application
6	Test SSO login

> **Warning:** Azure Entra has a **150-group limit** per SAML token. If a user belongs to more than 150 groups, the token will contain a group overage claim instead of individual group IDs. Always filter group claims to include only Dynatrace-relevant groups.

3.3 DNS Domain Validation

Dynatrace requires domain validation for SSO:

Step	Action
1	Add your email domain in Dynatrace Account Management
2	Add the provided TXT record to your DNS
3	Wait for DNS propagation (up to 48 hours)
4	Verify domain in Dynatrace Account Management

3.4 Test SSO Login

Before proceeding, verify SSO works end-to-end:

1. Open an incognito/private browser window
2. Navigate to `https://{tenant-id}.apps.dynatrace.com`
3. Click "Sign in with SSO"
4. Authenticate via your IdP
5. Verify you land in the Dynatrace tenant with correct permissions

> **Tip:** Test with at least one user from each IAM group to confirm group-based policy assignment works correctly.

3.5 Initial IAM Groups and Policies

Create the IAM structure designed in Step 3. At minimum:

Group	Policy	Purpose
`migration-admins`	Account admin + Environment admin	Migration team full access
`platform-admins`	Environment admin	Day-to-day administration
`sre-team`	Environment editor	SRE operations access
`viewers`	Environment viewer	Read-only stakeholders

> **Note:** These are starting groups. Refine IAM after migration is complete in Step 6 (Integrate). The goal here is to have enough access for migration activities.

---

4. ActiveGate Provisioning

Deploy new SaaS-connected ActiveGates **in parallel** with your existing Managed ActiveGates. This ensures continuous monitoring during migration and provides an easy rollback path.

4.1 Parallel Deployment Strategy

Phase	Managed AGs	SaaS AGs	OneAgents Point To
Pre-migration (now)	Running	Deploying	Managed AGs
During migration	Running	Running	Gradually shifting to SaaS AGs
Post-migration	Decommissioning	Running	SaaS AGs

4.2 ActiveGate Installation

Install ActiveGates connected to the SaaS tenant. Use the sizing from your Design step.

```
# Download ActiveGate installer from SaaS tenant
curl -o Dynatrace-ActiveGate-Linux.sh \
  "https://{tenant-
id}.live.dynatrace.com/api/v1/deployment/installer/gateway/unix/latest" \
  -H "Authorization: Api-Token {paas-token}"

# Install with network zone assignment
sudo /bin/sh Dynatrace-ActiveGate-Linux.sh \
  --set-network-zone={zone-name}
```

> **Deprecation note (SaaS 1.343, July 2026):** this classic ActiveGate deployment API (`/api/v1/deployment/installer/gateway/...`) is **deprecated** in favor of the Latest Dynatrace deployment REST API (platform-token support with fine-grained OAuth scopes; GA/enabled-by-default status arrives with the staged SaaS 1.343 tenant

> rollout from mid-July 2026 — verify in your tenant). The classic endpoint still works during the deprecation period — use it for in-flight migrations, but point new automation at the platform API.

4.3 Network Zone Assignment

Assign each AG to the correct network zone during installation:

```
# Assign during installation (recommended)
sudo /bin/sh Dynatrace-ActiveGate-Linux.sh --set-network-zone=datacenter-east

# Or assign after installation (ActiveGate 1.333+)
sudo agctl network-zone set datacenter-east
# ...or in /var/lib/dynatrace/gateway/config/custom.properties:
#   [connectivity]
#   networkZone = datacenter-east
# Then restart: sudo systemctl restart dynatracegateway
```

4.4 ActiveGate Groups

Create AG groups as designed in Step 3:

Group	Purpose	Configuration
`production-routing`	Route production OneAgent traffic	Default group for production zones
`nonprod-routing`	Route non-production traffic	Default group for non-production zones
`extensions`	Extensions 2.0 execution	Host-based AGs only
`synthetic`	Private synthetic monitoring	Dedicated synthetic AGs

> **Reminder:** Plan host-based ActiveGates for Extensions 2.0. The documented Kubernetes path covers SQL extensions run through the Dynatrace Operator (M2S-03 § 3.4).

4.5 Validate AG Connectivity

After deploying each ActiveGate, verify it appears in the SaaS tenant and is healthy.

```
// Verify ActiveGates connected to SaaS tenant
smartscapeNodes "ACTIVEGATE"
| fields name, version = dt.active_gate.version, zone = dt.network_zone.id,
is_containerized
| sort name asc

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
// ActiveGate 1.343 (published 07/15/2026, staged tenant rollout from
07/28/2026) deprecates
// GET /api/v2/activeGates, /api/v2/activeGates/{agId} and
/api/v2/activeGates/groups in favour of
// this same Smartscape node — the classic entity and the classic REST
endpoints were retired as one
// move. If you need a REST surface in the meantime, the classic Entities API
v2 selector
// (GET /api/v2/entities?entitySelector=type("ENVIRONMENT_ACTIVE_GATE")) is a
different surface and
// may still respond; the DQL `fetch dt.entity.*` form does not.
```

```
// Verify ActiveGate distribution across network zones
smartscapeNodes "ACTIVEGATE"
| summarize agCount = count(), by:{dt.network_zone.id}
| sort agCount desc

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
```

4.6 Connectivity Validation Checklist

Check	Command	Expected Result
AG → SaaS	`curl -v https://{tenant-id}.live.dynatrace.com` from AG host	HTTP 200
AG appears in tenant	DQL query above returns the AG	AG listed with correct zone
AG version current	Compare version to latest available	Within 1–2 minor versions
AG health	Dynatrace UI > Deployment status	Green / Healthy

---

5. SaaS Upgrade Assistant Setup

The SaaS Upgrade Assistant is a Dynatrace app that automates configuration migration from Managed to SaaS. Install and connect it now so it is ready for Step 5 (Execute).

5.1 Prerequisites

Requirement	Detail
Managed cluster version	No fixed minimum in the SaaS Upgrade Assistant docs; align to the same *major* version as the target SaaS tenant, and verify the current requirement in docs
Version alignment	Export from the same **major** version as the target SaaS environment to avoid false-positive migration failures (docs example: Managed 1.284.x → SaaS 1.284.x)
IAM policy	`upgrade-assistant:environments:write` (operate the app) + `app-engine:apps:install` (install the Hub app) assigned to migration users
Token scopes	`Read network zones`, `Write network zones`, `Capture request data` — added when the app gained network-zone and request-attribute support

5.2 Installation Steps

Step	Action
1	Open the Dynatrace Hub in your **SaaS tenant**
2	Search for "SaaS Upgrade Assistant"
3	Install the app
4	Grant the required IAM policy to migration users
5	Open the app and follow the connection wizard

5.3 Export the Managed Configuration and Upload It

The Upgrade Assistant does not connect to the Managed cluster. It works from a **configuration archive** that you export from the Cluster Management Console and upload to the app:

Step	Action
1	Sign in to the Managed **Cluster Management Console**
2	Go to **Environments** and select the environment to migrate

Step	Action
3	Select **Export configuration**, confirm, and store the archive locally
4	In the SaaS tenant, open the SaaS Upgrade Assistant and import the archive
5	Review the configurations it lists, grouped by type

> **Sources:** SaaS Upgrade Assistant (DT Managed docs) — export from the CMC *Environments* page, then import, review, edit and deploy in the app; SaaS Upgrade Assistant (Dynatrace Hub) — *"the required token's scope was extended to Read network zones, Write network zones and Capture request data"*.

5.4 Verify the Import

After the upload, confirm the app lists configurations for the environment you exported, and that the archive came from a Managed version aligned with the SaaS tenant (§ 5.1).

> **Tip:** Upload and review now, but do **not** deploy any configurations yet. Configuration migration happens in Step 5. Export a fresh archive just before Step 5 if anything changed on Managed since — the configuration freeze (§ 7) is what keeps that delta small.

5.5 Review Configuration Scope

The SaaS Upgrade Assistant groups configurations by type. Review the counts to validate against your Discovery inventory from Step 1:

Configuration Type	Expected Count (from Step 1)	SUA Count	Match?
Management zones			[ ]
Auto-tagging rules			[ ]
Dashboards			[ ]
Alerting profiles			[ ]
Request attributes			[ ]

Configuration Type	Expected Count (from Step 1)	SUA Count	Match?
Service detection rules			[ ]
SLOs			[ ]

> **Cluster-level safeguards:** in community practice, cluster overload-prevention settings — the maximum entry-point PurePaths/traces per process per minute and the maximum user actions per minute (RUM) — are reported as not carried by the export. Check whether they appear in your archive's review screen; if not, re-apply the values you inventoried in Step 1 manually on the SaaS tenant.

---

6. Network Zones in the SaaS Tenant

Network zones are among the configuration types the SaaS Upgrade Assistant migrates — its Hub listing names *network zones* in the supported list. Either way, every zone your agents will use must exist in the SaaS tenant, with SaaS ActiveGates assigned to it, **before** you redirect any OneAgent. Confirm the zones arrived with the archive; create any that did not.

> **Sources:** SaaS Upgrade Assistant (Dynatrace Hub) — lists *network zones* among the configurations the app supports.

6.1 What Has to Be True Before Agents Move

Check	Detail
Zones exist in SaaS	Imported by the Upgrade Assistant or created by hand
SaaS ActiveGates assigned	Managed ActiveGates do not connect to the SaaS tenant — the new ones carry the zones
Alternative zones and fallback mode set	Decide where traffic goes if a zone's ActiveGates are down (M2S-03 § 2)

Check	Detail
Names match	OneAgents keep their `--set-network-zone` value across the redirect, so SaaS zone names must match the Managed ones unless you reassign agents

6.2 Create Zones in SaaS

Create any zone the archive did not carry. Use the Settings UI or API:

Via Settings UI:

Settings > Network zones > Add network zone

> **API 1.339 deprecation (May 2026):** the `/api/v2/networkZones` endpoints are deprecated in favor of the Settings 2.0 schema `builtin:networkzones.zones`. Prefer the Settings 2.0 form below for new automation; the legacy endpoint still works during the deprecation period.

Via Settings 2.0 API (recommended):

```
# Create a network zone via Settings 2.0
curl -X POST "https://{tenant-id}.live.dynatrace.com/api/v2/settings/objects"
\
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:networkzones.zones",
    "scope": "environment",
    "value": {
      "id": "datacenter-east",
      "description": "Primary datacenter in East region",
      "alternativeZones": ["datacenter-west"],
      "fallbackMode": "ANY_ACTIVE_GATE"
    }
  }]'
# id, alternativeZones (may be empty) and fallbackMode are required. Needs
settings.write.
```

Via legacy Configuration API (deprecated, still functional during deprecation window):

```
# Create (or update) a network zone — PUT to the zone's ID; needs
networkZones.write
curl -X PUT "https://{tenant-
id}.live.dynatrace.com/api/v2/networkZones/datacenter-east" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '{
    "description": "Primary datacenter in East region",
    "alternativeZones": ["datacenter-west"]
  }'
```

> **Sources:** Network zones - Settings API (DT docs) — `fallbackMode` *"Valid values: ANY_ACTIVE_GATE (default), ONLY_DEFAULT_ZONE, NONE"*; PUT a network zone (DT docs) — *"This API is deprecated. Use the Settings API instead."*; Configure ActiveGate (DT docs) — `networkZone` *"Defines the network zone to which the ActiveGate belongs."*

6.3 Assign ActiveGates to Zones

If you deployed AGs with `--set-network-zone` during installation (Section 4), they are already assigned. Otherwise, update the assignment:

```
# Update AG network zone after installation (ActiveGate 1.333+)
sudo agctl network-zone set datacenter-east
# ...or in /var/lib/dynatrace/gateway/config/custom.properties:
#   [connectivity]
#   networkZone = datacenter-east
sudo systemctl restart dynatracegateway
```

6.4 Verify Zone Topology

```
// Verify network zone configuration and AG assignment
smartscapeNodes "ACTIVEGATE"
| fields name, zone = dt.network_zone.id
| sort zone asc, name asc

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
```

6.5 Zone Topology Validation

Compare the SaaS zone topology against your Design step:

Zone Name	Alternative Zone	AGs Assigned	Matches Design?
			[]
			[]
			[]
			[]

> **Important:** Every network zone must have at least 2 ActiveGates for high availability. Verify this before proceeding.

---

7. Configuration Freeze

Announce and enforce a configuration freeze on the Managed environment. This prevents configuration drift between the source (Managed) and target (SaaS) during migration.

7.1 What to Freeze

Category	Freeze Scope
Dashboards	No new dashboards or modifications to existing ones
Alerting	No new alerting profiles, metric events, or notification rules
Settings	No changes to management zones, auto-tagging, or service detection
Monitoring config	No changes to OneAgent features, deep monitoring, or anomaly detection
RUM/Mobile	No new web or mobile application definitions

7.2 Exception Process

Some changes may be unavoidable during the freeze period:

Exception Type	Approval Required	Documentation
Critical fix	Migration lead	Document change and apply to both Managed and SaaS
Security patch	Security team + Migration lead	Apply to both environments
New deployment	Migration lead	Ensure monitoring config exists in both environments

7.3 Communication Template

Send to all Dynatrace users and configuration owners:

> **Subject:** Dynatrace Configuration Freeze — Effective {date}

> As part of the Managed-to-SaaS migration, a configuration freeze is now in effect on the Managed environment. **Do not create or modify** dashboards, alerting rules, management zones, or any other configuration in the Managed tenant.
>
> All configuration changes during this period require approval from the migration lead. Contact {migration-lead-email} for exception requests.
>
> The freeze will remain in effect until the migration is complete (estimated {end-date}).

7.4 Freeze Verification

Monitor for unauthorized changes during the freeze period. The check runs in two places, with two different tools.

On Managed (the frozen source), use the audit log API, not DQL. Grail and DQL are not available on Managed. Configuration changes are recorded in the Environment API v2 audit log under the `CONFIG` category:

```
# Managed: configuration changes in the last 24 h
curl -s -G "https://{managed}/e/{env-id}/api/v2/auditlogs" \
  --data-urlencode 'filter=category("CONFIG")' \
  --data-urlencode 'from=now-24h' \
  -H "Authorization: Api-Token {MANAGED_TOKEN}"
```

The token needs the `auditLogs.read` scope.

On SaaS (the target), use DQL. Changes made directly in the SaaS tenant after the Upgrade Assistant run drift away from what was migrated. Settings changes are recorded as platform audit events in `dt.system.events`:

> **Sources:** Audit logs API - GET audit log (DT Managed docs) — *"you need an access token with auditLogs.read scope"*; Audit logs (DT docs).

```
// SaaS target: configuration changes in the last 24 h (Settings audit events)
// Do not run this on Managed — Grail and DQL are SaaS-only; use the audit log
API shown above there
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| fields timestamp, event.type, user.id, schema =
details.dt.settings.schema_id
| sort timestamp desc
| limit 50
```

---

8. Rollback Procedure

Before executing the migration, document and test a rollback procedure. If issues arise during Step 5, you need to be able to revert OneAgents back to Managed quickly.

8.1 Rollback Approach

Rollback redirects OneAgents from the SaaS tenant back to the Managed environment:

Component	Rollback Action
OneAgent	Redirect to Managed server URL
ActiveGate	No change needed (Managed AGs still running during dual-run)
Configuration	No change needed (Managed config is frozen, not deleted)
Data	Managed retains all historical data until decommissioned

8.2 OneAgent Rollback Command

The key rollback command redirects the OneAgent back to Managed:

```
# Linux rollback — server, environment ID and tenant token change together
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl \
  --set-server="{managed-server-url}" \
  --set-tenant={managed-env-id} \
  --set-tenant-token={managed-tenant-token} \
  --restart-service

# Windows rollback
"C:\Program Files\dynatrace\oneagent\agent\tools\oneagentctl.exe" ^
  --set-server="{managed-server-url}" ^
  --set-tenant={managed-env-id} ^
  --set-tenant-token={managed-tenant-token} ^
  --restart-service
```

Changing `--set-server` alone is not enough: the agent would keep the SaaS environment ID and token and fail to authenticate against Managed. Dynatrace's own guidance for `--set-tenant` is *"Always use in combination with --set-tenant-token"*. Take the exact server URL from the Managed OneAgent deployment page, or from `oneagentctl --get-server` on a host before you redirect it.

> **Sources:** OneAgent configuration via command-line interface (DT docs) — *"Always use in combination with --set-tenant-token"*.

8.3 Pre-Migration Rollback Test

Test rollback on a non-production host **before** the production migration:

Step	Action	Validation
1	Select a non-production host currently reporting to Managed	Host visible in Managed UI
2	Redirect OneAgent to SaaS: `oneagentctl --set-server=... --set-tenant={tenant-id} --set-tenant-token=... --restart-service` (M2S-05 § 4)	Host appears in SaaS UI within 5 minutes
3	Verify monitoring data flows to SaaS	Metrics, logs, spans visible in SaaS
4	Execute rollback with the full command in § 8.2	Host reappears in Managed UI

Step	Action	Validation
5	Verify monitoring data resumes in Managed	Metrics, logs, spans visible in Managed

> **Important:** Do not proceed to Step 5 until you have successfully tested rollback on at least one host. This validates that the Managed environment is still fully operational and ready to accept agents back if needed.

8.4 Document Rollback Details

Record these values for use during migration:

Parameter	Value
Managed server URL	`{managed-server-url}` (copy the exact value from `oneagentctl --get-server`)
Managed environment ID	`{managed-env-id}`
Managed tenant token	(retrieve from Managed UI > Deployment > OneAgent)
SaaS server URL	`https://{tenant-id}.live.dynatrace.com/communication`
SaaS environment ID	`{tenant-id}`
SaaS tenant token	(retrieve from SaaS UI > Deployment > OneAgent)
Rollback tested on	{hostname}, {date}
Rollback time	~5 minutes for agent to reconnect

---

9. Readiness Checklist

Complete every checkpoint before proceeding to Step 5 (Execute). Each item corresponds to a section in this notebook.

Licensing

Checkpoint	Status
Dual-run licensing confirmed with Dynatrace account team	[ ]
DPS consumption model understood and budgeted	[ ]
Migration timeline communicated to account team	[ ]

SaaS Tenant

Checkpoint	Status
SaaS tenant provisioned in correct data residency region	[ ]
Tenant accessible at `{tenant-id}.live.dynatrace.com`	[ ]
Tenant accessible at `{tenant-id}.apps.dynatrace.com`	[ ]
Baseline settings configured (timezone, currency, retention)	[ ]
Grail bucket strategy designed before ingest (buckets, retention, query-billing) — see ORGNZ-03 / OPLOGS-04	[ ]
API access verified	[ ]

SSO and Identity

Checkpoint	Status
SAML SSO configured and tested	[ ]
IdP signs full SAML message (Entra: *Sign SAML response and assertion*)	[ ]
Azure Entra group claims filtered (if applicable)	[ ]
DNS domain validated	[ ]
Initial IAM groups and policies created	[ ]
SSO tested with users from each IAM group	[ ]

ActiveGates

Checkpoint	Status
New SaaS-connected AGs deployed per Design step sizing	[ ]
Each AG assigned to correct network zone	[ ]
Minimum 2 AGs per network zone for HA	[ ]
AG groups created (routing, extensions, synthetic)	[ ]
AG connectivity to SaaS validated	[ ]
AG health confirmed in Dynatrace UI	[ ]

SaaS Upgrade Assistant

Checkpoint	Status
App installed in SaaS tenant	[ ]
`upgrade-assistant:environments:write` IAM policy granted	[ ]
Managed configuration archive exported from the CMC	[ ]
Archive imported and reviewed in the app	[ ]
Configuration counts match Discovery inventory	[ ]

Network Zones

Checkpoint	Status
All zones present in SaaS (imported by the Upgrade Assistant or created)	[ ]
Alternative zones configured for failover	[ ]
AGs assigned to correct zones	[ ]
Zone topology matches Design step	[ ]

Configuration Freeze

Checkpoint	Status
Configuration freeze announced to all stakeholders	[ ]
Exception process documented and communicated	[ ]
Monitoring for unauthorized changes in place	[ ]

Rollback

Checkpoint	Status
Managed server URL, environment ID and tenant token documented	[ ]
SaaS server URL, environment ID and tenant token documented	[ ]
Rollback procedure documented	[ ]
Rollback tested on non-production host	[ ]
Rollback test successful (agent reconnected to Managed)	[ ]

Communication

Checkpoint	Status
Migration schedule communicated to stakeholders	[ ]
Support team briefed on rollback procedure	[ ]
Escalation contacts identified for migration window	[ ]

Next Step

> → **M2S-05: Step 5 — Execute** — Migrate configurations using the SaaS Upgrade Assistant and redirect OneAgents from Managed to SaaS.

Continue the Series

Next Notebook	Focus
M2S-05: Step 5 — Execute	Configuration migration and agent redirection

Preparation Resources

- SaaS Upgrade Assistant Documentation
- SaaS Upgrade Assistant on Dynatrace Hub
- ActiveGate Installation
- Network Zones Configuration
- SAML SSO Setup
- IAM Documentation
- OneAgent Communication
- Azure Native Dynatrace Service

Summary

In this notebook, you prepared:

- **Licensing** — Dual-run licensing coordinated with Dynatrace account team for the overlap period
- **SaaS tenant** — Provisioned in the correct data residency region with baseline configuration
- **SSO and identity** — SAML 2.0 configured, DNS domain validated, initial IAM groups created
- **ActiveGates** — New SaaS-connected AGs deployed in parallel, assigned to network zones
- **SaaS Upgrade Assistant** — Installed, Managed configuration archive exported and imported for review
- **Network zones** — All zones present in SaaS (imported or created) matching the Design step topology
- **Configuration freeze** — Announced and enforced on the Managed environment

- **Rollback** — Procedure documented and tested on a non-production host

> **Key Takeaway:** Preparation is the most critical step in a smooth migration. Every item in the readiness checklist reduces risk during execution. Do not proceed to Step 5 until every checkbox is complete—especially the rollback test. A tested rollback gives confidence to move forward.

---

*Continue to **M2S-05: Step 5 — Execute** to begin migrating configurations and redirecting agents.*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.